

使用 Kustomize 調度資源

來源：<https://codelabs.developers.google.com/scaling-with-kustomize?hl=zh-tw>

步驟數：6

目錄

1. [1. 目標](#)
2. [2. 準備工作區](#)
3. [3. 使用 kustomize 指令列用戶端](#)
4. [4. 覆寫常見元素](#)
5. [5. 修補較大的 YAML 結構](#)
6. [6. 使用多個疊加圖層](#)

1. 目標

Kustomize 是一種工具，可讓您以不使用範本的方式自訂應用程式設定，簡化現成應用程式的使用方式。這個公用程式可獨立使用，也可透過 `kubectl apply -k` 建構至 `kubectl`，或做為獨立的 CLI 使用。詳情請參閱 kustomize.io。

在本教學課程中，您將瞭解 Kustomize 的一些核心概念，並使用這項工具管理應用程式和環境的變化。

您將學會以下內容：

- 使用 `kustomize` 指令列用戶端
 - 覆寫常見元素
 - 修補較大的 YAML 結構
 - 使用多個疊加圖層
-

步驟 2 · 約 0 分鐘

2. 準備工作區

1. 前往下列網址開啟 Cloud Shell 編輯器

```
https://ide.cloud.google.com
```

1. 在終端機視窗中，為本教學課程建立工作目錄

```
mkdir kustomize-lab
```

1. 切換至目錄並設定 IDE 工作區

```
cd kustomize-lab && cloudshell workspace .
```

3. 使用 kustomize 指令列用戶端

kustomize 的強大之處在於能夠以自訂值疊加及修改基本 Kubernetes YAML。為此，kustomize 需要含有檔案位置 and 要覆寫內容指示的基礎檔案。Kustomize 屬於 Kubernetes 生態系統，可透過多種方法執行。

在本節中，您將建立基本 kustomize 設定，並使用獨立的 kustomize 指令列用戶端處理檔案。

1. 首先，請建立資料夾來存放基本設定檔

```
mkdir -p chat-app/base
```

2. 在基本資料夾中建立簡單的 Kubernetes `deployment.yaml`

```
cat <<EOF > chat-app/base/deployment.yaml
```

```
kind: Deployment
```

```
apiVersion: apps/v1
```

```
metadata:
```

```
name: app
```

```
spec:
```

```
template:
```

```
EOF
```

```
`metadata:`  
  
  `name: chat-app`  
  
`spec:`  
  
  `containers:`  
  
    - name: chat-app`  
  
      `image: chat-app-image`
```

```
EOF
```

3. 建立基礎 `kustomization.yaml`

Kustomize 會尋找名為 `kustomization.yaml` 的檔案做為進入點。這個檔案包含對各種基本和覆寫檔案的參照，以及特定覆寫值。

建立 `kustomization.yaml` 檔案，將 `deployment.yaml` 參照為基本資源。

```
cat <<EOF > chat-app/base/kustomization.yaml
```

```
bases:
```

```
- deployment.yaml
```

```
EOF
```

4. 在基礎資料夾上執行 `kustomize` 指令。這麼做會輸出部署作業 YAML 檔案，但不會有任何變更，這是預期行為，因為您尚未納入任何變化版本。

```
kustomize build chat-app/base
```

這個獨立用戶端可以與 `kubectl` 用戶端合併，直接套用輸出內容，如下列範例所示。這樣做會將建構指令的輸出內容直接串流至 `kubectl apply` 指令。

(Do Not Execute - Included for reference only)

```
kustomize build chat-app/base | kubectl apply -f -
```

如果需要特定版本的 `kustomize` 用戶端，這個方法就非常實用。

或者，您也可以使用 `kubectl` 內建的工具執行 `kustomize`。如下列範例所示。

(Do Not Execute - Included for reference only)

```
kubectl apply -k chat-app/base
```

4. 覆寫常見元素

現在您已設定工作區，並驗證 kustomize 正常運作，接下來要覆寫一些基本值。

圖片、命名空間和標籤通常會針對每個應用程式和環境進行自訂。由於這些值經常變更，Kustomize 可讓您直接在 `kustomize.yaml` 中宣告這些值，不必為這些常見情況建立許多修補程式。

這項技術通常用於建立範本的特定執行個體。現在只要變更名稱和命名空間，即可將一組基本資源用於多項實作。

在本範例中，您將新增命名空間、名稱前置字串，並為 `kustomization.yaml` 新增一些標籤。

1. 更新 `kustomization.yaml` 檔案，加入常見標籤和命名空間。

在終端機中複製並執行下列指令

```
cat <<EOF > chat-app/base/kustomization.yaml
```

```
bases:
```

```
- deployment.yaml
```

```
namespace: my-namespace
```

```
nameprefix: my-
```

```
commonLabels:
```

```
app: my-app
```

```
EOF
```

2. 執行建構指令

此時執行建構作業會顯示，產生的 YAML 檔案現在包含服務和部署作業定義中的命名空間、標籤和加上前置字元的名稱。

```
kustomize build chat-app/base
```

請注意，輸出內容包含部署作業 YAML 檔案中沒有的標籤和命名空間。請注意，名稱也從「`chat-app`」變更為「`my-chat-app`」

(請勿複製輸出內容)

```
kind: Deployment
```

```
metadata:
```

```
labels:
```

```
  app: my-app`
```

```
name: my-chat-app
```

```
namespace: my-namespace
```

步驟 5 · 約 0 分鐘

5. 修補較大的 YAML 結構

Kustomize 也提供套用修補程式的功能，可覆蓋基本資源。這項技術通常用於提供應用程式和環境之間的變異性。

在這個步驟中，您將為使用相同基礎資源的單一應用程式建立環境變異版本。

1. 首先，請為不同環境建立資料夾

```
mkdir -p chat-app/dev
```

```
mkdir -p chat-app/prod
```

2. 使用下列指令編寫階段修補程式

```
cat <<EOF > chat-app/dev/deployment.yaml
```

```
kind: Deployment
```

```
apiVersion: apps/v1
```

```
metadata:
```

```
name: app
```

```
spec:
```

```
template:
```

```
`spec:`  
  
  `containers:`  
  
    `- name: chat-app`  
  
      `env:`  
  
        `- name: ENVIRONMENT`  
  
          `value: dev`
```

```
EOF
```

3. 現在使用下列指令編寫 prod 修補程式

```
cat <<EOF > chat-app/prod/deployment.yaml
```

```
kind: Deployment
```

```
apiVersion: apps/v1
```

```
metadata:
```

```
name: app
```

```
spec:
```

```
template:
```



```
`spec:`  
  
  `containers:`  
  
    `- name: chat-app`  
  
      `env:`  
  
        `- name: ENVIRONMENT`  
  
          `value: prod`
```

EOF

請注意，上述修補程式不含容器映像檔名稱。該值位於您在上一步建立的 `base/deployment.yaml` 中。不過，這些修補程式確實包含開發和實際工作環境的專屬環境變數。

4. 為基本目錄實作 kustomize YAML 檔案

重新編寫基礎 `kustomization.yaml`，移除命名空間和名稱前置字串，因為這只是基礎設定，沒有任何變化。這些欄位稍後會移至環境檔案。

```
cat <<EOF > chat-app/base/kustomization.yaml
```

```
bases:
```

```
- deployment.yaml
```

```
commonLabels:
```

```
app: chat-app
```

EOF

5. 為開發目錄實作 kustomize YAML 檔案

現在，請在終端機中執行下列指令，為開發和正式環境實作變體。

```
cat <<EOF > chat-app/dev/kustomization.yaml
```

```
bases:
```

```
- ../base
```

```
namespace: dev
```

```
nameprefix: dev-
```

```
commonLabels:
```

```
env: dev
```

```
patches:
```

```
- deployment.yaml
```

EOF

請注意檔案中新增加的 `patches`：區段。這表示 `kustomize` 應將這些檔案疊加在基本資源上。

6. 為 prod 目錄實作 kustomize YAML 檔案

```
cat <<EOF > chat-app/prod/kustomization.yaml
```

```
bases:
```

```
- ../base
```

```
namespace: prod
```

```
nameprefix: prod-
```

```
commonLabels:
```

```
env: prod
```

```
patches:
```

```
- deployment.yaml
```

```
EOF
```

7. 執行 kustomize 合併檔案

建立基礎和環境檔案後，即可執行 kustomize 程序來修補基礎檔案。

執行下列指令，即可查看合併結果。

```
kustomize build chat-app/dev
```

請注意，輸出內容包含合併結果，例如來自基礎和開發設定的標籤，以及來自基礎的容器映像檔名稱和來自開發資料夾的環境變數。

6. 使用多個疊加圖層

許多機構都有專責團隊，負責支援應用程式團隊及管理平台。這些團隊通常會希望所有環境的所有應用程式中加入特定詳細資料，例如記錄代理程式。

在本範例中，您將建立 `shared-kustomize` 資料夾和資源，這些資源會包含在所有應用程式中，且與應用程式部署的環境無關。

1. 建立 shared-kustomize 資料夾

```
mkdir shared-kustomize
```

2. 在共用資料夾中建立簡單的 deployment.yaml

```
cat <<EOF > shared-kustomize/deployment.yaml
```

```
kind: Deployment
```

```
apiVersion: apps/v1
```

```
metadata:
```

```
name: app
```

```
spec:
```

```
template:
```

```
`spec:`  
  
  `containers:`  
  
    - name: logging-agent`  
  
    `image: logging-agent-image`
```

```
EOF
```

3. 在共用資料夾中建立 kustomization.yaml

```
cat <<EOF > shared-kustomize/kustomization.yaml
```

```
bases:
```

```
- deployment.yaml
```

```
EOF
```

4. 從應用程式參照 shared-kustomize 資料夾

由於您希望 `shared-kustomize` 資料夾做為所有應用程式的基礎，因此需要更新 `chat-`

`app/base/kustomization.yaml`，將 `shared-kustomize` 做為基礎。然後在頂端修補自己的 `deployment.yaml`。環境資料夾隨後會再次修補。

在終端機中複製並執行下列指令

```
cat <<EOF > chat-app/base/kustomization.yaml
```

```
bases:
- ../../shared-kustomize

commonLabels:

app: chat-app

patches:
- deployment.yaml

EOF
```

5. 執行 kustomize 並查看開發環境的合併結果

```
kustomize build chat-app/dev
```

請注意，輸出內容包含應用程式基礎、應用程式環境和 `shared-kustomize` 資料夾的合併結果。具體來說，您可以在容器部分看到所有三個位置的值。

(請勿複製輸出內容)

<pre>

```
`containers:`

  `- env:`

    `- name: ENVIRONMENT`

      `value: dev`

    `name: chat-app`

  `- image: image`

    `name: app`

  `- image: logging-agent-image`

    `name: logging-agent`
```

</pre>
